

Глава 1 Обобщение – Основи на обучението с подкрепление

Изследване върху възможностите за прилагане на изкуствен интелект в компютърните игри

Alexander Andreev

April 2026

Съдържание

1 Глава 1 – Основи на обучението с подкрепление	1
1.1 Как обучението с подкрепление се утвърди като самостоятелна дисциплина	1
1.2 Основни принципи – агент, действие, среда	2
1.3 Формулировка на MDP	3
1.4 Информационни множества – когато не можете да видите всичко	4
1.5 Динамично програмиране – при наличие на модел	5
1.6 Обучение с темпорална разлика – обучение без модел	6
1.7 DQN – Дълбоки Q-мрежи	7
1.8 PPO – оптимизация на стратегията с ограничение на близостта	8
1.9 Практическа проверка – собствени реализации срещу Stable-Baselines3	10

1 Глава 1 – Основи на обучението с подкрепление

1.1 Как обучението с подкрепление се утвърди като самостоятелна дисциплина

Обучението с подкрепление не възниква изведнъж като завършена дисциплина. Неговите корени се простират в множество области, които дълги десетилетия почти не са били свързани помежду си.

Най-ранната нишка идва от животинската психология в началото на XX век – „Законът за ефекта“ на Торндайк (1911)¹ предполага, че действията, последвани от удовлетворяващи резултати, стават по-вероятни. Това всъщност е идеята за сигнал за награда и началото на нишката на обучението чрез проба и грешка. Успоредна нишка преминава през теорията на оптималното управление през 50-те години на миналия век, където Белман разработва динамично програмиране за решаване на последователни проблеми на вземане на решение.² Той въвежда функцията на стойността и принципа на оптималност, които и двете са в ядрото на съвременното обучение с подкрепление.

¹Thorndike, E.L. (1911). *Animal Intelligence: Experimental Studies*. Macmillan.

²Bellman, R. (1957). *Dynamic Programming*. Princeton University Press.

Трета, по-слабо обособена нишка са методите на темпоралната разлика: програмата за дама на Самюел (1959)³ се учи, като играе срещу себе си, и приближава оценката на всяка позиция към оценките на следващите позиции.

Тези нишки се сливат в края на 80-те години на миналия век. Сътън (1988)⁴, продължавайки докторската си работа от 1984 г., отделя обучението с темпорална разлика от задачата за управление и го представя като общ метод за прогнозиране. Q-обучението на Уоткинс (1989)⁵ окончателно обединява нишките на темпоралната разлика и на оптималното управление и дава на областта първия ясен безмоделен алгоритъм за управление; сходимостта му е доказана през 1992 г. Към момента, когато Сътън и Барто публикуват своя учебник през 1998 г. (актуализиран през 2018 г.),⁶ обучението с подкрепление вече се е утвърдило като разпознаваема дисциплина със собствена нотация, таксономия на проблемите и алгоритмичен инструментариум.

Революцията в дълбокото обучение достига до обучението с подкрепление през 2013–2015 г., когато DQN на DeepMind се научава да играе игри на Atari само по суровите пиксели на екрана.⁷ Тази демонстрация ясно показва, че алгоритмите за обучение с подкрепление, комбинирани с невронни апроксиматори на функции, могат да решават високоизмерни проблеми от реалния свят. PPO се появява през 2017 г. като практичен метод на градиента на стратегията⁸ и оттогава обучението с подкрепление се разширява в областите на роботиката, игровия изкуствен интелект, системите за препоръки и съгласуването на езиковите модели с човешките предпочитания (RLHF).

Обучението с подкрепление не е просто „машинно обучение с награди“, а самостоятелна рамка за последователно вземане на решения при несигурност, със собствени теоретични основи, произтичащи от теорията на управлението, психологията и статистиката.⁶

1.2 Основни принципи – агент, действие, среда

Цялата рамка на обучението с подкрепление се основава на прост цикъл. **Агентът** наблюдава текущото **състояние** на **средата**, избира **действие** и получава сигнал за **награда** заедно с ново **състояние**. След това процесът се повтаря. Целта на агента е да научи **стратегия** – съпоставяне между състояния и действия – която максимизира натрупаната награда във времето.

³Samuel, A.L. (1959). “Some Studies in Machine Learning Using the Game of Checkers.” *IBM Journal of Research and Development*, 3(3), 210-229.

⁴Sutton, R.S. (1988). “Learning to Predict by the Methods of Temporal Differences.” *Machine Learning*, 3(1), 9-44. Докторска дисертация: Sutton, R.S. (1984). *Temporal Credit Assignment in Reinforcement Learning*. University of Massachusetts Amherst.

⁵Watkins, C.J.C.H. (1989). *Learning from Delayed Rewards*. Докторска дисертация, University of Cambridge. Сходимостта е доказана в Watkins, C.J.C.H. & Dayan, P. (1992). “Q-learning.” *Machine Learning*, 8(3-4), 279-292.

⁶Sutton, R.S. & Barto, A.G. (2018). *Reinforcement Learning: An Introduction*, второ издание. MIT Press. Гл. 1 “Introduction”; гл. 3 “Finite Markov Decision Processes”; гл. 4 “Dynamic Programming”; гл. 5 “Monte Carlo Methods”; гл. 6 “Temporal-Difference Learning”; §11.3 “The Deadly Triad”; гл. 13 “Policy Gradient Methods”. <http://incompleteideas.net/book/the-book-2nd.html>

⁷Mnih, V. et al. (2013). “Playing Atari with Deep Reinforcement Learning.” arXiv:1312.5602; Mnih, V. et al. (2015). “Human-level control through deep reinforcement learning.” *Nature*, 518(7540), 529-533. DOI 10.1038/nature14236

⁸Schulman, J., Wolski, F., Dhariwal, P., Radford, A. & Klimov, O. (2017). “Proximal Policy Optimization Algorithms.” arXiv:1707.06347.

Няколко уточнения относно терминологията:

- **Състояние** (s): описание на средата в даден момент. В CartPole това включва позицията на количката, скоростта на количката, ъгъла на пръта и ъгловата скорост на пръта. В покера биха били раздадените карти и историята на залозите.
- **Действие** (a): възможните действия на агента. Те могат да бъдат дискретни (например бутане наляво или надясно) или непрекъснати (например прилагане на конкретен въртящ момент).
- **Награда** (r): скаларна величина за обратна връзка, която агентът получава след всяко действие. Наградите могат да бъдат редки (само в края на играта) или чести (на всеки времеви интервал).
- **Стратегия** (π): правилото, по което агентът избира действията си. Тя може да бъде детерминистична ($\pi(s) = a$) или стохастична, като $\pi(a|s)$ дава разпределение на вероятностите върху възможните действия.
- **Функция на стойността** ($V^\pi(s)$): очакваната натрупана награда от състояние s нататък, ако агентът следва стратегия π . Тя показва на агента колко добро е едно състояние в дългосрочен план, а не само непосредствено.
- **Коефициент на дисконтиране** (γ): число между 0 и 1, което определя доколко агентът отчита бъдещите награди спрямо непосредствените такива. Висока стойност на $\gamma = 0,99$ означава, че агентът е търпелив; ниска стойност на $\gamma = 0,5$ показва, че е късоглед.

Критичното предизвикателство при проектирането на системи за обучение с подкрепление е **компромисът изследване–експлоатация**. Агентът трябва да изследва непознати действия, за да открие потенциално по-добри стратегии, но същевременно и да експлоатира наличните си знания, за да събира награда. Прекомерното изследване води до загуба на време, а прекалената експлоатация може да доведе до заседналост в локален оптимум.

Съществуват два основни класа алгоритми. **Методите, базирани на стойност** (като DQN⁷) научават стойността на състоянията или двойките състояние–действие и извеждат стратегия въз основа на тези стойности. **Методите, базирани на стратегията** (като PPO⁸), научават самата стратегия директно.⁶ И двата подхода имат своите предимства; значителна част от областта на обучението с подкрепление е посветена на тяхното ефективно комбиниране.

Прочетете повече: OpenAI Spinning Up – „Part 1: Key Concepts in RL“

https://spinningup.openai.com/en/latest/spinningup/rl_intro.html

1.3 Формулировка на MDP

Марковски процес на вземане на решения (Markov decision process, MDP) представлява формалната математическа рамка, върху която са изградени повечето алгоритми за обучение с подкрепление. Той включва пет основни компонента:

- S : множество от състояния
- A : множество от действия
- $P(s'|s, a)$: функция на преход – вероятност за преминаване в състояние s' при дадено състояние s и действие a
- $R(s, a, s')$: функция на наградата
- $\gamma \in [0, 1)$: коефициент на дисконтиране

Частта „Марков“ означава, че бъдещето зависи единствено от текущото **състояние**, а не от историята на пътя до него. Това е силно предположение – и такова, което не е валидно в много интересни ситуации (като **покер**, където историята на залозите има значение). Разделът „Информационни множества – когато не можете да видите всичко“ по-долу разглежда този въпрос.

Целта е да се намери стратегия π^* , която максимизира очакваната дисконтирана възвръщаемост.

$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right]$$

Ключовата рекурсивна зависимост е **уравнението на Белман**:

$$V^{\pi}(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^{\pi}(s')]$$

Това означава, че стойността на едно състояние е равна на очакваната непосредствена награда плюс дисконтираната стойност на следващото състояние. Динамичното програмиране, обучението с темпорална разлика (temporal-difference learning, TD) и дълбокото обучение с подкрепление се основават на вариации на това уравнение.⁶

1.4 Информационни множества – когато не можете да видите всичко

Стандартните MDP предполагат, че агентът може напълно да наблюдава състоянието. В действителност обаче често това не е така. В покера например не виждате картите на опонента, а в игрите със стратегия в реално време действа т.нар. **мъгла на войната**. Това са **частично наблюдаеми** среди.

Формалното разширение е **частично наблюдаемият марковски процес на вземане на решения** (Partially Observable MDP, POMDP)⁹, който включва функция за наблюдение – агентът получава наблюдения, представляващи шумни или непълни изгледи на истинското състояние. Но POMDP са печално известни с това, че оптималното им решаване е изключително трудно.

⁹Kaelbling, L.P., Littman, M.L. & Cassandra, A.R. (1998). “Planning and acting in partially observable stochastic domains.” *Artificial Intelligence*, 101(1-2), 99-134. DOI 10.1016/S0004-3702(98)00023-X

В теорията на игрите същата идея се изразява чрез **информационни множества**. Информационно множество обединява всички състояния на играта, които играчът не може да различи едно от друго въз основа на направените наблюдения. В Кун покер (3 карти, 2 играчи), когато държите Вале и все още не е имало залагания, вие се намирате в информационно множество – знаете своята карта, но не и тази на опонента си, така че не можете да определите дали действителното състояние е „Аз имам Вале, опонентът има Дама“ или „Аз имам Вале, опонентът има Поп“.

Алгоритмите, които работят в тези среди – като CFR (минимизиране на контрафактичното съжаление, разгледано в глава 2) – оперират върху информационни множества, а не върху отделни състояния. Това представлява фундаментално различен изчислителен проблем от стандартното RL и е причината игровият изкуствен интелект да се нуждае от собствен набор инструменти, различни от тези, които DQN или PPO предоставят по подразбиране.

Усвояването на информационните множества е от значение, защото главите от 2 до 8 се занимават с игри с непълна информация. Алгоритмите се променят, но **ядрото** на концепцията остава: вие вземате решения въз основа на това, което можете да наблюдавате, като разсъждавате върху това, което не можете.¹⁰

1.5 Динамично програмиране – при наличие на модел

Динамичното програмиране (DP) представлява най-ранният подход за решаване на MDP. То е приложимо, когато разполагате с пълна информация за средата: вероятностите за преход $P(s'|s, a)$ и функцията на наградата $R(s, a, s')$. На практика това означава, че правилата на играта са ви напълно известни.

Двата основни алгоритъма при динамичното програмиране са:

Итерация по стратегии редува два етапа. Първо се извършва *оценка на стратегия*: при дадена фиксирана стратегия се изчислява стойността на всяко състояние чрез многократно прилагане на уравнението на Белман, докато не се постигне сходимост. След това следва *подобрене на стратегията*: стратегията се актуализира така, че да бъде алчна спрямо новоизчислените стойности. Процесът се повтаря, докато стратегията престане да се променя. Гарантирано е, че процесът достига оптималната стратегия.

Итерация на стойностите е съкратен вариант. Вместо да се оцени напълно една **стратегия**, преди тя да бъде подобрена, тя комбинира оценката и подобренето в една-единствена актуализация.

¹⁰Shoham, Y. & Leyton-Brown, K. (2008). *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Cambridge University Press. DOI 10.1017/CBO9780511811654. Гл. 3 “Introduction to Noncooperative Game Theory: Games in Normal Form”; гл. 4 “Computing Solution Concepts of Normal-Form Games” (§4.1 – линейната програма за игри на двама с нулева сума); гл. 5 “Games with Sequential Actions: Reasoning and Computing with the Extensive Form” (§5.2 – игри с непълна информация и информационни множества; §5.2.3 – последователната форма, механизмът под всяка линейна програма в Глава 8); гл. 7 “Learning and Teaching” – рамката за учене в повтарящи се игри, включително напрежението, че действията едновременно *експлоатират* и *обучават* опонента. Свободно достъпна: <http://www.masfoundations.org/download.html>

На всяка стъпка, за всяко **състояние**, се избира **действието**, което максимизира очакваната едностъпкова възвръщаемост плюс дисконтираната стойност на следващото **състояние**. Тя клони към оптималната функция на стойността, от която се извлича оптималната **стратегия**.

DP е полезен, защото осигурява точни решения и има ясни гаранции за сходимост. Недостатъкът е очевиден: необходим е пълният модел, а изчислението нараства с броя на състоянията. За MDP с около дванадесет състояния DP е тривиален. За играта Го със 10^{170} състояния това е невъзможно.

Причината DP да има значение, въпреки че рядко го използваме директно, е, че всеки следващ метод – методът Монте Карло, TD обучението, DQN, PPO – може да се разглежда като приближение към решението на DP, когато моделът е неизвестен или пространството на състоянията е твърде голямо. DP представлява теоретичния таван, към който практическите алгоритми се стремят да се доближат.⁶

1.6 Обучение с темпорална разлика – обучение без модел

Обучението с темпорална разлика (TD) е пробивът, който прави обучението с подкрепление приложимо на практика. За разлика от динамичното програмиране, то не изисква модел на средата. За разлика от методите на Монте Карло, то не е необходимо да чака края на един епизод, за да се учи. То актуализира стойностните оценки след всяка стъпка, използвайки самоподкрепяне: текущата оценка на стойността на следващото състояние замества истинската бъдеща възвръщаемост.

Основната актуализация на TD(0) за функцията на стойността е:

$$V(s_t) \leftarrow V(s_t) + \alpha [r_t + \gamma V(s_{t+1}) - V(s_t)]$$

Изразът в скобите е **TD грешката** – разликата между настъпилото (награда плюс оценена бъдеща стойност) и предвиденото. Ако TD грешката е положителна, състоянието се е оказало по-добро от очакваното; ако е отрицателна, то е било по-лошо.

Вариантът на тази идея за задачата за управление е **Q-обучението** (Watkins, 1989):⁵

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right]$$

Q-обучението научава стойността на двойки състояние-действие и използва оператора $\max$, затова се обучава **извън текущата стратегия** (off-policy): научава оптималната стратегия дори докато следва друга, експериментална стратегия. Това го прави много практично.

TD обучението има предимство пред динамичното програмиране, защото не изисква модел, а пред метода Монте Карло – защото се учи онлайн (на всяка стъпка, а не на всеки **епизод**) и има

по-ниска **дисперсия**. Самоподкрепянето въвежда известно отместване (bias), но на практика този компромис е силно изгоден. TD методите са основата, върху която се градят DQN и повечето съвременни **алгоритми, базирани на стойности**.

SARSA е аналогът на Q-обучението, който се обучава по текущата стратегия (on-policy) – той актуализира стойностите, използвайки действието, което агентът действително предприема след това, а не най-доброто възможно действие. Това го прави по-консервативен и понякога по-безопасен на практика, но не му позволява да научи оптимална стратегия, докато следва неоптимална изследователска стратегия.⁶

1.7 DQN – Дълбоки Q-мрежи

DQN (Mnih и др., 2015)⁷ е алгоритъмът, който доказва възможността дълбоките невронни мрежи да се използват като апроксиматори в обучението с подкрепление в голям мащаб. Преди DQN обучението с Q-стойности беше ограничено до таблична среда или ръчно извлечени признаци. DQN показва, че може да се научи да играе 49 игри на Atari директно по суровите пиксели на екрана, като надмина човешкото представяне в много от тях.

Как работи. DQN заменя Q-таблицата с невронна мрежа, която приема състояние като вход и извежда Q-стойности за всички действия. Обучението на мрежата се извършва чрез минимизиране на квадратичната TD грешка – разликата между предсказаната Q-стойност и целта (награда плюс дисконтираната максимална Q-стойност на следващото състояние). На всяка стъпка агентът избира действието с най-висока Q-стойност (експлоатация) или произволно действие с вероятност ϵ (изследване).

Два ключови похвата осигуряват достатъчна стабилност, за да работи този подход:

1. **Повторение на натрупан опит.** Вместо да се обучава върху последователни преходи (които са силно корелирани), преходите се съхраняват в голям буфер и се извличат на случаен принцип в минипартиди. Това нарушава корелацията, намалява дисперсията и позволява всеки опит да бъде използван многократно. Принципът е аналогичен на разбъркването на тренировъчните данни при обучение с учител.
2. **Целева мрежа.** Поддържа се отделно копие на Q-мрежата, което се обновява само периодически (на всеки няколко хиляди стъпки или епизода). Целевата стойност на TD се изчислява с помощта на това замразено копие. Без него целта се измества при всяка тренировъчна стъпка – вие преследвате подвижна цел, което води до колебания и дори до разходимост на обучението. Целевата мрежа разделя процеса на обновяване и стабилизира обучението.

Защо DQN е важен в сравнение с предишните методи: табличното Q-обучение не може да се справи с високоизмерни състояния (изображения, непрекъснати наблюдения); апроксимацията на функция само с Q-обучение беше известна като нестабилна („смъртоносна триада“⁶: обучение извън текущата стратегия + апроксимация на функции + самоподкрепяне). Двата похвата

на DQN – повторението на натрупан опит и целевата мрежа – ограничиха нестабилността и за първи път направиха практически приложимо дълбокото обучение с подкрепление, базирано на стойности.

Ограничения. DQN работи единствено с дискретни пространства на действията. Той може да надценява Q-стойностите (проблем, който Double DQN¹¹ решава). Тъй като се обучава извън текущата стратегия, той използва данните ефективно, но това може да доведе до остаряла информация в буфера за повторение на опита. И най-важното – той научава алчна детерминистична стратегия, без естествен начин за изразяване на стохастични стратегии, което е от значение в игрите, където смесените стратегии са оптимални.

В нашата реализация от Глава 1 DQN реши CartPole-v1 (средна награда за последните 100 епизода 477,5 при цел 475) след приблизително 1000 епизода обучение. Най-влиятелните хиперпараметри бяха размерът на мрежата, епсилон минимумът и честотата на синхронизация на целевата мрежа.

Бележка относно невронните мрежи в тази глава. Q-мрежата в DQN е стандартен многослоен перцептрон (MLP), използван тук като практичен инструмент – функция, която съпоставя на състоянията Q-стойности. За Глава 1 вътрешните механизми на невронната мрежа (инициализация на теглата, потокът на градиента, функциите на активиране) бяха разглеждани като черна кутия: използвахме настройките по подразбиране на PyTorch и се фокусирахме върху самия RL алгоритъм. По-задълбочено разглеждане на архитектурите на невронните мрежи и тяхното взаимодействие с методите за намиране на равновесие ще бъде представено в Глава 5 (Невронни мрежи за игри с непълна информация).

1.8 PPO – оптимизация на стратегията с ограничение на близостта

PPO (Schulman и др., 2017)⁸ прилага фундаментално различен подход към RL в сравнение с DQN. Вместо да изчислява стойността на действията и да извежда стратегия въз основа на тези стойности, PPO **научава стратегията директно** – невронна мрежа генерира разпределение на вероятностите върху действията, а мрежата се обучава да увеличава вероятността на действията, които водят до висока възвръщаемост.

Проблемът, който PPO решава. Методите на градиента на стратегията са мощни в теорията, но са печално известни с нестабилността си на практика. Основният алгоритъм REINFORCE¹² страда от висока дисперсия и може да предприема разрушително големи стъпки, които провалят добра стратегия. Оптимизацията на стратегията в доверителна област (Trust Region Policy Optimization, TRPO; Schulman и др., 2015)¹³ се справи с това, като ограничи колко може да се промени стратегията-

¹¹van Hasselt, H., Guez, A. & Silver, D. (2016). “Deep Reinforcement Learning with Double Q-Learning.” *Proceedings of the AAAI Conference on Artificial Intelligence*, 30(1). DOI 10.1609/aaai.v30i1.10295. arXiv:1509.06461

¹²Williams, R.J. (1992). “Simple statistical gradient-following algorithms for connectionist reinforcement learning.” *Machine Learning*, 8(3-4), 229-256. DOI 10.1007/BF00992696

¹³Schulman, J., Levine, S., Abbeel, P., Jordan, M. & Moritz, P. (2015). “Trust Region Policy Optimization.” *Proceedings of the 32nd International Conference on Machine Learning*, 1889-1897.

та при всяка актуализация, но TRPO трябва да решава ограничена задача от втори ред (спрегнати градиенти върху матрицата на Фишер за ограничението по KL и последващо линейно търсене), което прави реализацията му сложна.

Как работи PPO. PPO запазва идеята за стабилност от TRPO, но заменя ограничението с много по-прост механизъм: **изрязване**. Ключовото количество е отношението на вероятностите $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{old}}(a_t|s_t)$ - колко по-вероятно (или по-малко вероятно) е новата стратегия да предприеме същото действие, което старата стратегия е предприела. PPO изрязва това отношение до диапазона $[1 - \epsilon, 1 + \epsilon]$ (обикновено $\epsilon = 0,2$), което означава:

- Ако едно **действие** се окаже добро (положително **предимство**) и отношението на вероятностите надхвърли $1 + \epsilon$, **градиентът** се анулира – **стратегията** не трябва да става прекалено „алчна“ за това **действие**.
- Ако едно **действие** се окаже лошо (отрицателно **предимство**) и отношението на вероятностите падне под $1 - \epsilon$, **градиентът** също се анулира – така се предотвратява прекомерна корекция на **стратегията**.

Това е **изрязаната сурогатна цел** и именно тя осигурява работата на PPO. Тя постига стабилност, сходна с тази на TRPO, само с обикновени стохастични градиентни стъпки от първи ред, което значително опростява реализацията.

Архитектура. PPO е актьор-оценител: **мрежа на стратегията** (актьор) извежда вероятности за действия, а **мрежа за стойности** (оценител) оценява стойностите на състоянията. Мрежата за стойности позволява прилагането на обобщената оценка на предимството (generalized advantage estimation, GAE)¹⁴, която дава оценки на предимството с ниска дисперсия за **градиента на стратегията**. И двете мрежи се обучават едновременно.

Защо PPO е важен в сравнение с DQN: PPO естествено работи както с дискретни, така и с непрекъснати пространства на **действията**. Той може да усвоява **стохастични стратегии**, което е от решаващо значение в игрови среди, където прилагането на **смесени стратегии** е необходимо. Тъй като се обучава **по текущата си стратегия** (on-policy) и използва само пресни данни, събрани с нея, той избягва проблемите с остарели данни. А изрязаната цел го прави забележително устойчив – PPO се превърна в един от най-широко използваните алгоритми на дълбокото обучение с подкрепление; с него например е реализиран първоначалният RLHF процес за езикови модели, които следват инструкции.¹⁵

Ограничения. Тъй като се обучава по текущата си стратегия, PPO използва данните по-малко ефективно от методите, които се обучават извън текущата стратегия (off-policy), като DQN (всеки опит се използва само за няколко **епохи на обновяване**, преди да бъде отхвърлен). Той изисква внимателна настройка на оценката на **предимството** (параметър λ на GAE), **бонуса за ен-**

¹⁴Schulman, J., Moritz, P., Levine, S., Jordan, M. & Abbeel, P. (2016). “High-Dimensional Continuous Control Using Generalized Advantage Estimation.” *ICLR 2016*. arXiv:1506.02438

¹⁵Ouyang, L., Wu, J., Jiang, X. et al. (2022). “Training language models to follow instructions with human feedback.” arXiv:2203.02155.

тропия и архитектурата на **мрежата**. Освен това все още може да среща трудности при редки награди – ако **агентът** рядко постига успех, наличният сигнал за учене е недостатъчен.

В нашата реализация от Глава 1 PPO успя да реши LunarLander-v3 (средна награда за последните 100 епизода 202,2 при цел 200) след приблизително 264 хиляди стъпки на средата. Решаващата промяна между двете записани изпълнения беше капацитетът на мрежата (удвоен от [64,64] на [128,128], при по-голям бюджет); нормализацията на предимството беше използвана и в двете.

Бележка относно невронните мрежи в тази глава. Както и при DQN, мрежите за **стратегия** и **стойност** тук са стандартни многослойни перцептрони (MLP), чиито вътрешни механизми не бяха във фокуса на тази глава. Приехме прагматичен подход (използвайки налични настройки за тях) към невронните компоненти и се концентрирахме върху разбирането на логиката на алгоритъма PPO (изрязване, GAE, структура на разгръщането). По-задълбоченото взаимодействие между дизайна на **невронната мрежа** и **сходимостта** в обучението с подкрепление – особено в контекста на **агенти за игри** – е отложено за Глава 5.

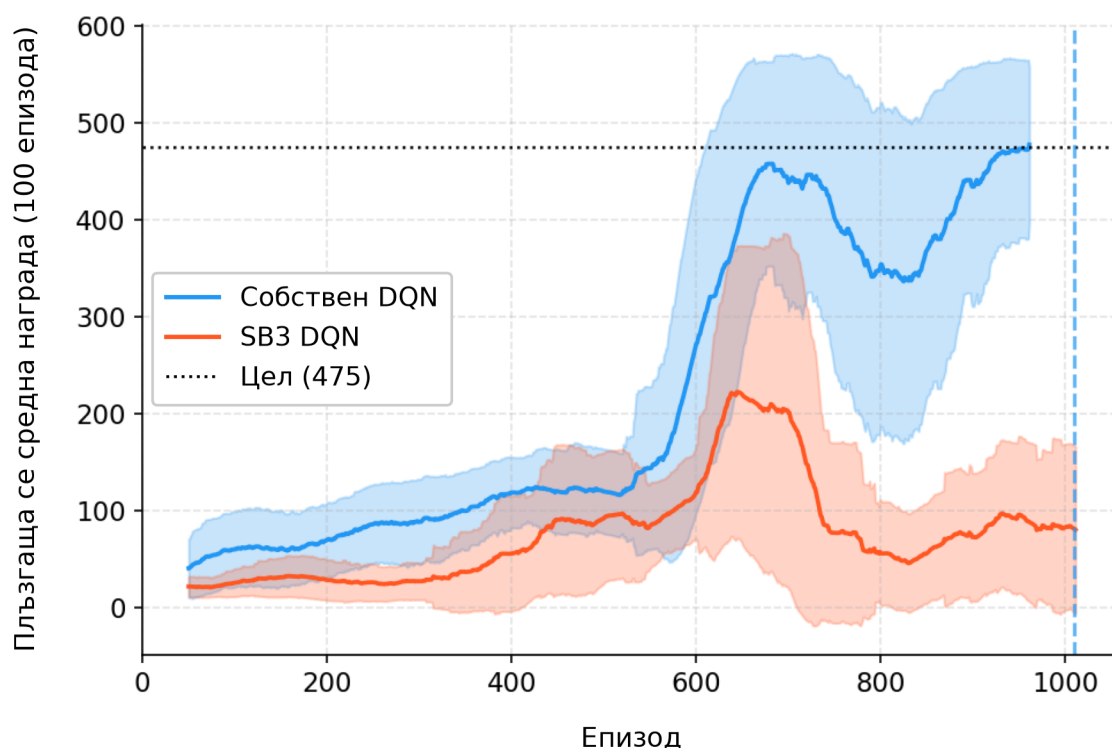
1.9 Практическа проверка – собствени реализации срещу Stable-Baselines3

За да проверим дали нашите реализации от нулата функционират коректно, ги сравнихме със Stable-Baselines3 (SB3)¹⁶ – широко използвана и добре тествана RL библиотека. При PPO на SB3 бяха зададени нашите хиперпараметри и същият бюджет от 500 хил. стъпки. При DQN показаното изпълнение на SB3 използва настроените за CartPole хиперпараметри от RL Baselines3 Zoo¹⁷ и бюджет от 100 хил. стъпки, така че там сравнението е с настроен базов агент, а не при еднакви настройки.

¹⁶Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M. & Dormann, N. (2021). “Stable-Baselines3: Reliable Reinforcement Learning Implementations.” *Journal of Machine Learning Research*, 22(268), 1-8.

¹⁷Raffin, A. (2020). *RL Baselines3 Zoo*. Хранилище в GitHub. <https://github.com/DLR-RM/rl-baselines3-zoo>; настройките за CartPole-v1 са в `hyperparams/dqn.yml`.

1.9.1 DQN върху CartPole-v1



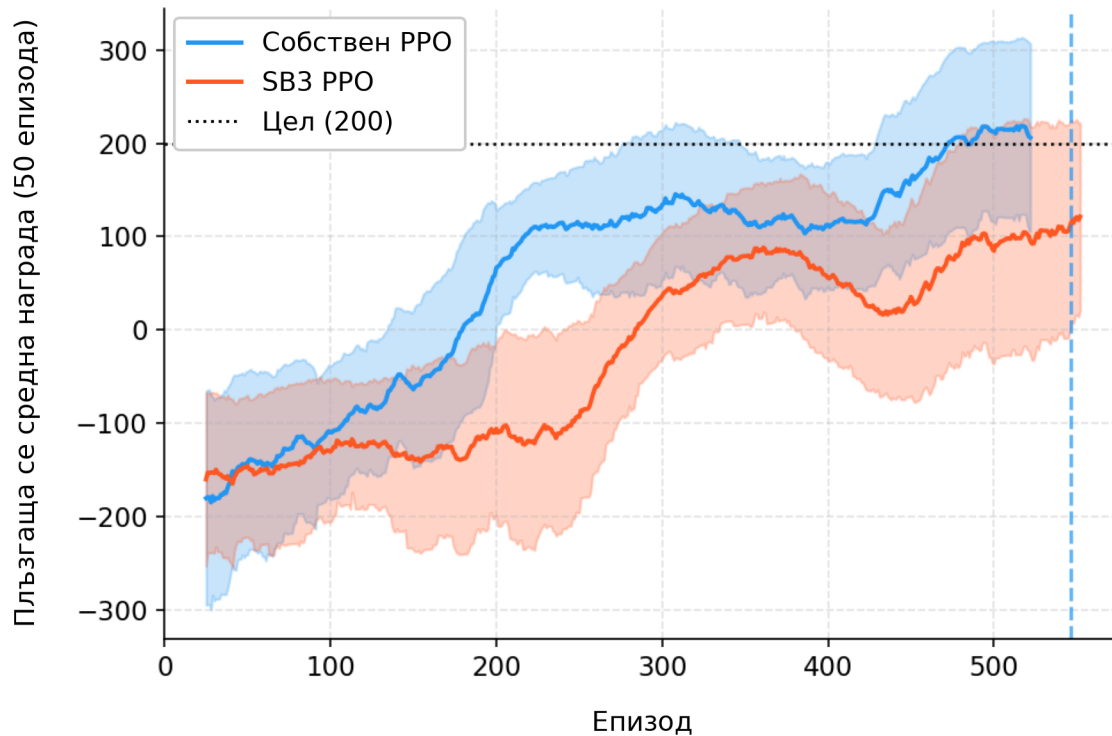
Фигура 1: DQN върху CartPole-v1: собствената реализация срещу SB3 (изпълнение от 6 април 2026 г.; показани са първите 1 061 от 1 255 епизода на SB3). Плъзгаща се средна на наградата за 100 епизода (оцветено – $\pm$ едно стандартно отклонение в прозореца); пунктирната линия е целта 475, вертикалната – крайт на обучението на собствения агент.

Нашият собствен DQN решава CartPole около епизод 1011, като достига плъзгаща се средна за 100 епизода 477,5 (над целта от 475). Показаното изпълнение на DQN на SB3 е от 6 април 2026 г. и наградите му по епизоди са в запазения кеш с резултати (`sb3_results_cache.json`). То използва настройките на Zoo (скорост на обучение 0,0023, мрежа [256,256], синхронизация на целевата мрежа на всеки 10 стъпки, ϵ намалява до 0,04 през първите 16 хил. стъпки) с бюджет от 100 хил. стъпки – два пъти повече от 50-те хиляди, за които Zoo ги обучава. През своите 1 255 епизода плъзгащата му се средна достига най-много 222,4 при епизод 695 (около 51 хил. стъпки) и завършва на 134,3, без да се доближи до целта.

Тъй като SB3 използва настроени хиперпараметри, а не същите като нашите, сравнението не е равностойно и причината за разликата не е установена. Две различия не са проверени. Първо, показаните награди са от епизоди на обучение, в които SB3 все още избира случайно действие в 4% от случаите (крайно $\epsilon = 0,04$), докато нашият ϵ е спаднал под 1% към момента на решаването на задачата. Второ, SB3 прави 128 градиентни стъпки на всеки 256 стъпки на средата, т.е. наполовина по-малко обновявания на стъпка от нашия агент, който обновява мрежата при всяка стъпка. Всяка реализация е изпълнена веднъж, нашата – без фиксирано начално число, така че неуспехът на SB3

при бюджет, два пъти по-голям от този на Zoo, трябва да се провери с второ начално число, преди да се тълкува като свойство на SB3, а не на това изпълнение.

1.9.2 PPO върху LunarLander-v3



Фигура 2: PPO върху LunarLander-v3: собствената реализация (окончателното изпълнение, спряно при епизод 547) срещу SB3 (показани са първите 577 от 1 224 епизода). Плъзгаща се средна на наградата за 50 епизода (оцветено – $\pm$ едно стандартно отклонение в прозореца); пунктирната линия е целта 200, вертикалната – края на обучението на собствения агент.

Нашият собствен PPO преминава целта от 200 около епизод 543 (след 264 хил. стъпки). PPO на SB3 със същия бюджет от 500 хил. стъпки достига най-добра плъзгаща се средна за 100 епизода 131,2 – все още се покачва и не е достигнал сходимост.

Разликата тук е по-малка, отколкото при DQN. И двата използват идентични хиперпараметри на PPO. Оставащата разлика може да се дължи на ортогоналната инициализация на теглата в SB3 и на това, че SB3 нормализира предимството във всяка мини-партида, а не в цялото разгръщане; при едно-единствено изпълнение на всяка реализация тя може да е и случайна. При предоставяне на по-голям обучителен бюджет (1–2 млн. стъпки), PPO на SB3 вероятно би достигнал целта.

1.9.3 Извод

Чиста реализация от нулата обхваща около 300–500 реда код, докато тази на SB3 достига приблизително 10 000. Компромисът е по-малко функции срещу пълна прозрачност и лесна възможност

за отстраняване на грешки. За изследователски цели – особено при планираната работа върху експлоатацията на противника в глави 7–8 – собствените реализации дават възможност за прецизни модификации (например добавяне на наблюдения на състоянието на убежденията), които биха изисквали значително подкласиране в SB3. Именно тази практическа гъвкавост беше причината да изградим решение от нулата.